claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_9b4c71d0-6a0\agent-a29dde5f2e1137256.jsonl`
- SHA-256: `f84308873cf1c8e2c561204a6360f01156e944d3d2e88937300ea2022b4c856e`
- Source modified: `2026-06-21T13:43:46+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_9b4c71d0-6a0`
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`

Transcript

1. user (2026-06-21T13:40:05.045Z)

Review the COMPUTE_DECOUPLED_SERVING M9 edge-auth change just committed on branch feat/serving-m9-auth (HEAD = 03077c3). Inspect with: git diff origin/master...HEAD and read: backend/api/auth.py (verify_cf_access_token + resolve_tenant + JWKS cache/fetch), backend/main.py (the import-time CORS-from-config + the /api/process tenant resolution), backend/config/models.py (SecurityConfig: edge_auth_enabled/cf_team_domain/cf_access_aud/cors_allow_origins), backend/infrastructure/database.py (create_job owner_id), tests/test_edge_auth.py, pyproject.toml ([dev]+[auth] PyJWT). CONTRACT: DEFAULT-OFF ? security.edge_auth_enabled=False ? resolve_tenant returns None (the local single-user, unchanged) EVEN with a token present; create_job owner_id NULL; CORS default ['*']. When ON, the Cf-Access JWT is REQUIRED + verified against the team JWKS (RS256: sig+exp+iss+aud) so a direct hit to the Cloud Run URL can't spoof the identity header (D9). `jwt` is lazy-imported; the JWKS fetch is injectable for offline tests. Report ONLY real, grounded issues with file:line. Empty findings if a lens finds nothing real.

LENS = TEST RIGOR / HYGIENE. (1) The reject-path tests (expired/wrong-aud/wrong-iss/bad-sig/unknown-kid/no-claim) ? does each actually fail for the REASON claimed, or could some pass for an unrelated reason (e.g. bad-sig test failing on kid instead of signature)? (2) test_bad_signature_rejected signs with `other` key but kid=test-kid (default) ? does it truly exercise signature failure against the real jwks? (3) The /api/process API tests: default-OFF owner None, missing-token 401, valid-token owner set ? are they non-vacuous (the 401 is from auth, not a 400/validation error)? Does the valid-token test correctly monkeypatch the JWKS fetcher so no network is hit? (4) Is there a test that a token with a DIFFERENT-but-valid signature from a key NOT in the JWKS is rejected (the core anti-spoof)? (5) ruff/mypy issues, the pytest.importorskip placement, the module-scoped keypair fixture reuse. Find vacuous/weak tests or hygiene gaps.

2. assistant (2026-06-21T13:40:07.390Z)

I'll review the M9 edge-auth change with a focus on test rigor and hygiene. Let me start by gathering the relevant files and the diff.

3. user (2026-06-21T13:40:09.128Z)

```
 backend/api/auth.py                | 127 ++++++++++++++++++++++
 backend/config/models.py           |  14 +++
 backend/infrastructure/database.py |  12 +--
 backend/main.py                    |  29 ++++--
 pyproject.toml                     |   5 ++
 tests/test_edge_auth.py            | 179 +++++++++++++++++++++++++++++++++
 6 files changed, 354 insertions(+), 12 deletions(-)
=== HEAD ===
03077c3 feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)
```

4. assistant (2026-06-21T13:40:11.151Z)

Let me read the diff and all the relevant files.

5. user (2026-06-21T13:40:11.728Z)

```
diff --git a/backend/api/auth.py b/backend/api/auth.py
new file mode 100644
index 0000000..eb187bc
--- /dev/null
+++ b/backend/api/auth.py
@@ -0,0 +1,127 @@
+"""Cloudflare Access edge-auth ? verify the Cf-Access JWT, derive the tenant ``user_id`` (M9,
D9).
+
+**DEFAULT-OFF:** when ``security.edge_auth_enabled`` is False the resolver is a strict NO-OP
(returns
+``None`` = the local single-user) ? nothing about the existing local path changes. When
enabled, the
+``Cf-Access-Jwt-Assertion`` header is **required** and **verified** (RS256) against the team
JWKS ?
+signature + exp + issuer + audience ? so a direct hit to the Cloud Run URL **can't spoof** the
+identity header (the explicit D9 anti-spoof requirement). Returns the verified ``user_id`` (the
+token's ``email``, else ``sub``).
+
+``jwt`` (``PyJWT[crypto]``) is **lazy-imported** so the base runtime works without it (install
the
+``auth`` extra for hosted/multi-tenant). The JWKS fetch is **injectable**, so the whole
verifier is
+unit-tested **offline** with an in-test RSA keypair + a fake JWKS ? no network, no Cloudflare.
+"""
+
+from __future__ import annotations
+
+import json
+import time
+import urllib.request
+from typing import Any, Callable, Mapping, Optional
+
+# Cloudflare Access sends the signed assertion in this header (it terminates at the CF edge).
+CF_ACCESS_HEADER = "Cf-Access-Jwt-Assertion"
+_JWKS_PATH = "/cdn-cgi/access/certs"
+_JWKS_TTL_SEC = 600.0  # re-fetch the team JWKS at most every 10 min (keys rotate slowly)
+
+JwksFetcher = Callable[[str], dict]
+
+
+class EdgeAuthError(Exception):
+    """A required Cf-Access token was missing or failed verification ? the caller returns
401/403."""
+
+
+# module-level JWKS cache: team_domain -> (fetched_at, jwks_dict). Production only; tests
inject.
+_jwks_cache: dict[str, tuple[float, dict]] = {}
+
+
+def _issuer(team_domain: str) -> str:
+    return team_domain.rstrip("/")
+
+
+def _default_jwks_fetcher(team_domain: str) -> dict:
+    """Fetch the team's public JWKS (``<team_domain>/cdn-cgi/access/certs``). Network ? only
the
+    production path reaches here; tests pass an injected fetcher."""
```

```
+    url = _issuer(team_domain) + _JWKS_PATH
+    with urllib.request.urlopen(url, timeout=5) as resp:  # noqa: S310 (https team domain)
+        data: Any = json.load(resp)
+    if not isinstance(data, dict) or "keys" not in data:
+        raise EdgeAuthError(f"unexpected JWKS shape from {url}")
+    return data
+
+
+def _get_jwks(team_domain: str, fetcher: Optional[JwksFetcher]) -> dict:
+    """Return the JWKS for ``team_domain``, cached with a TTL. An injected ``fetcher`` (tests)
+    bypasses the cache so each test is deterministic."""
+    if fetcher is not None:
+        return fetcher(team_domain)
+    now = time.time()
+    cached = _jwks_cache.get(team_domain)
+    if cached and (now - cached[0]) < _JWKS_TTL_SEC:
+        return cached[1]
+    jwks = _default_jwks_fetcher(team_domain)
+    _jwks_cache[team_domain] = (now, jwks)
+    return jwks
+
+
+def verify_cf_access_token(token: str, *, team_domain: str, aud: str, jwks: dict) -> str:
+    """Verify a Cf-Access JWT against ``jwks`` (RS256; checks sig + exp + issuer + audience)
and
+    return the user_id (``email`` else ``sub``). Raises ``EdgeAuthError`` on ANY failure."""
+    try:
+        import jwt
+        from jwt import PyJWKSet
+    except Exception as exc:  # pragma: no cover - only when the auth extra isn't installed
+        raise EdgeAuthError(
+            "PyJWT[crypto] is required for edge auth ? install the 'auth' extra "
+            "(pip install -e .[auth]) on a host with edge_auth_enabled."
+        ) from exc
+
+    try:
+        kid = jwt.get_unverified_header(token).get("kid")
+        if not kid:
+            raise EdgeAuthError("token header has no kid")
+        signing_key = PyJWKSet.from_dict(jwks)[kid]
+        claims = jwt.decode(
+            token, signing_key.key, algorithms=["RS256"],
+            audience=aud, issuer=_issuer(team_domain),
+            options={"require": ["exp", "iss", "aud"]},
+        )
+    except EdgeAuthError:
+        raise
+    except KeyError as exc:  # kid not in the JWKS
+        raise EdgeAuthError(f"no JWKS key for kid {exc}") from exc
+    except Exception as exc:  # jwt.* errors: bad sig / expired / wrong aud|iss / malformed
+        raise EdgeAuthError(f"Cf-Access token verification failed: {exc}") from exc
+
+    user = claims.get("email") or claims.get("sub")
+    if not user:
+        raise EdgeAuthError("verified token carries no email/sub claim")
+    return str(user)
+
+
+def resolve_tenant(headers: Mapping[str, str], config: Any, *,
+                   jwks_fetcher: Optional[JwksFetcher] = None) -> Optional[str]:
+    """Resolve the tenant ``user_id`` for a request, or ``None`` when edge auth is OFF.
+
+    DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-user;
unchanged).
+    Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403) on any
```

```
miss.
+    ``jwks_fetcher`` is injectable for offline tests.
+    """
+    sec = getattr(config, "security", None)
+    if not sec or not getattr(sec, "edge_auth_enabled", False):
+        return None  # DEFAULT-OFF ? no tenant identity, byte-identical to today
+
+    team_domain = getattr(sec, "cf_team_domain", "") or ""
+    aud = getattr(sec, "cf_access_aud", "") or ""
+    if not team_domain or not aud:
+        raise EdgeAuthError(
+            "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not configured")
+
+    # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but support a
plain
+    # dict too (tests) by trying both casings.
+    token = headers.get(CF_ACCESS_HEADER) or headers.get(CF_ACCESS_HEADER.lower())
+    if not token:
+        raise EdgeAuthError(f"missing {CF_ACCESS_HEADER} header (request did not pass
Cloudflare Access)")
+
+    jwks = _get_jwks(team_domain, jwks_fetcher)
+    return verify_cf_access_token(token, team_domain=team_domain, aud=aud, jwks=jwks)
diff --git a/backend/config/models.py b/backend/config/models.py
index 71cdc0e..4f27623 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -293,6 +293,20 @@ class SecurityConfig(BaseModel):
     max_part_mb: int = 95
     allowed_upload_extensions: List[str] = ["mp4", "mov"]

+    # --- M9 edge auth (Cloudflare Access, D9) ---
+    # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local single-user
+    # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-Jwt-
Assertion`
+    # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
+    # header) and tags the job with the verified user_id (owner_id). The team domain (e.g.
+    # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is the CF
Access
+    # application's audience tag. NO secrets here ? these are public identifiers.
+    edge_auth_enabled: bool = False
+    cf_team_domain: str = ""        # e.g. https://<team>.cloudflareaccess.com (issuer + /certs
JWKS)
+    cf_access_aud: str = ""         # the CF Access application AUD tag the token must carry
+    # CORS allow-origins. Default ["*"] keeps the local web UI working; tighten to the hosted
+    # origin(s) in multi-tenant mode. (credentials stay OFF ? Cf-Access is a header, not a
cookie.)
+    cors_allow_origins: List[str] = ["*"]
+

 class StorageConfig(BaseModel):
     """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
diff --git a/backend/infrastructure/database.py b/backend/infrastructure/database.py
index 6dede93..4e0ac4a 100644
--- a/backend/infrastructure/database.py
+++ b/backend/infrastructure/database.py
@@ -538,15 +538,17 @@ class Database:
     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
                    player_pool: Optional[List[str]] = None,
                    max_frames: Optional[int] = None,
-                   policy: Optional[Dict[str, Any]] = None) -> bool:
+                   policy: Optional[Dict[str, Any]] = None,
+                   owner_id: Optional[str] = None) -> bool:
         """Insert a new job in state 'queued'. ``policy`` is an optional JSON ExecutionPolicy
```

```
-        (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
+        (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
+        (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default)."""
         return self._execute_write(
-            "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames, status,
policy) "
-            "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
+            "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames, status,
policy, "
+            "owner_id) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?)",
             (job_id, video_path, segmenter,
              json.dumps(player_pool) if player_pool is not None else None,
-             max_frames, json.dumps(policy) if policy is not None else None),
+             max_frames, json.dumps(policy) if policy is not None else None, owner_id),
             f"Failed to create job {job_id}",
         )

diff --git a/backend/main.py b/backend/main.py
index ef1588f..c0be6b1 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -16,7 +16,7 @@ logging.basicConfig(
     datefmt='%H:%M:%S'
 )
 logger = logging.getLogger(__name__)
-from fastapi import FastAPI, HTTPException
+from fastapi import FastAPI, HTTPException, Request
 from fastapi.middleware.cors import CORSMiddleware
 from pydantic import BaseModel
 from typing import List, Dict, Any, Optional
@@ -40,15 +40,22 @@ from backend.results import Failure  # standardized error/result contract
 from backend.reporting import emit_indexer_report
 from backend import storage  # M2: URI-aware input acquisition (local = identity passthrough)
 from backend import billing  # M8: per-job cost ledger (USD internal / INR display)
+from backend.api import auth as edge_auth  # M9: Cloudflare Access edge-auth (default-OFF)

 app = FastAPI(title="Sports Highlight Indexer API")

-# Enable CORS for the local web interface. allow_origins='*' with allow_credentials=True is
-# rejected by browsers per the fetch spec and is an unsafe default to carry into the
-# auth/tenancy work; this tool uses no cookies, so credentials stay off.
+# CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by browsers
per the
+# fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so credentials
stay
+# off. M9: the origin list is configurable (security.cors_allow_origins, default ["*"]) so a
hosted
+# deploy can lock it to its origin. Read at import (a cheap, torch-free config load; defaults
when no
+# config.json) ? the middleware is fixed at startup.
+try:
+    _cors_origins = load_app_config().security.cors_allow_origins
+except Exception:
+    _cors_origins = ["*"]
 app.add_middleware(
     CORSMiddleware,
-    allow_origins=["*"],
+    allow_origins=_cors_origins,
     allow_credentials=False,
     allow_methods=["*"],
     allow_headers=["*"],
@@ -266,7 +273,7 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
```

```
 @app.post("/api/process", status_code=202)
-def process_video(req: ProcessRequest):
+def process_video(req: ProcessRequest, request: Request):
     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
@@ -274,6 +281,14 @@ def process_video(req: ProcessRequest):
     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
     """
+    # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local single-user,
+    # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a missing
or
+    # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
+    try:
+        owner_id = edge_auth.resolve_tenant(request.headers, global_config)
+    except edge_auth.EdgeAuthError as e:
+        raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
+
     allowed_root = getattr(getattr(global_config, "security", None), "allowed_video_root",
None)
     try:
         validate_input_path(req.video_path, allowed_root=allowed_root)
@@ -299,7 +314,7 @@ def process_video(req: ProcessRequest):

     job_id = uuid.uuid4().hex
     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
-                                  req.player_pool, req.max_frames):
+                                  req.player_pool, req.max_frames, owner_id=owner_id):
         raise HTTPException(status_code=500, detail="Failed to persist job record.")
     _get_executor().submit(_drain_due_jobs)
     return JSONResponse(
diff --git a/pyproject.toml b/pyproject.toml
index ec11f3f..282bc83 100644
--- a/pyproject.toml
+++ b/pyproject.toml
@@ -44,8 +44,13 @@ dev = [
     "httpx",        # fastapi TestClient transport
     "ruff",
     "mypy",
+    "PyJWT[crypto]>=2.8",  # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
 ]

+# M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when
security.edge_auth_enabled=True.
+# `jwt` is lazy-imported, so the base runtime works without this; install it for hosted/multi-
tenant.
+auth = ["PyJWT[crypto]>=2.8"]
+
 # Per-video observability reports (SOFT dependency: the pipeline degrades to a
 # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
 # fetched over SSH; if you lack access, simply omit this group ? reporting
diff --git a/tests/test_edge_auth.py b/tests/test_edge_auth.py
new file mode 100644
index 0000000..fbe7d25
--- /dev/null
+++ b/tests/test_edge_auth.py
@@ -0,0 +1,179 @@
+"""Cloudflare Access edge-auth (COMPUTE_DECOUPLED_SERVING M9, D9).
+
+Fully OFFLINE: an in-test RSA keypair signs Cf-Access JWTs + builds a fake JWKS, so the
verifier +
+the tenant resolver are exercised with no network and no Cloudflare. Cardinal invariant: with
+``edge_auth_enabled=False`` the resolver is a strict NO-OP (returns None = today's local
```

```
single-user)
+EVEN when a token is present.
+"""
+import json
+import time
+
+import pytest
+
+pytest.importorskip("jwt")  # PyJWT[crypto] (in the [dev] extra); skip cleanly where absent
+import jwt  # noqa: E402
+from jwt.algorithms import RSAAlgorithm  # noqa: E402
+from cryptography.hazmat.primitives.asymmetric import rsa  # noqa: E402
+
+from backend.api import auth as edge_auth  # noqa: E402
+from backend.config import AppConfig  # noqa: E402
+
+TEAM = "https://team.cloudflareaccess.com"
+AUD = "test-aud-tag"
+KID = "test-kid"
+
+
+@pytest.fixture(scope="module")
+def keypair():
+    """An RSA private key + a JWKS carrying its public key (kid=test-kid)."""
+    key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
+    jwk = json.loads(RSAAlgorithm.to_jwk(key.public_key()))
+    jwk.update({"kid": KID, "alg": "RS256", "use": "sig"})
+    return key, {"keys": [jwk]}
+
+
+def _token(key, *, email="u@example.com", aud=AUD, iss=TEAM, exp_delta=3600, kid=KID,
sub=None):
+    now = int(time.time())
+    claims = {"aud": aud, "iss": iss, "exp": now + exp_delta, "iat": now}
+    if email:
+        claims["email"] = email
+    if sub:
+        claims["sub"] = sub
+    return jwt.encode(claims, key, algorithm="RS256", headers={"kid": kid})
+
+
+def _cfg(enabled, *, team=TEAM, aud=AUD):
+    return AppConfig.model_validate({"security": {
+        "edge_auth_enabled": enabled, "cf_team_domain": team, "cf_access_aud": aud}})
+
+
+# ---------------------------------------------------------------- verifier (accept)
+
+def test_valid_token_returns_email(keypair):
+    key, jwks = keypair
+    assert edge_auth.verify_cf_access_token(
+        _token(key), team_domain=TEAM, aud=AUD, jwks=jwks) == "u@example.com"
+
+
+def test_falls_back_to_sub_when_no_email(keypair):
+    key, jwks = keypair
+    tok = _token(key, email=None, sub="user-123")
+    assert edge_auth.verify_cf_access_token(tok, team_domain=TEAM, aud=AUD, jwks=jwks) ==
"user-123"
+
+
+# ---------------------------------------------------------------- verifier (reject paths)
+
+@pytest.mark.parametrize("kw", [
+    {"exp_delta": -10},                    # expired
```

```
+    {"aud": "wrong-aud"},                    # wrong audience (not this CF app)
+    {"iss": "https://evil.example"},         # wrong issuer
+    {"kid": "unknown-kid"},                  # kid not in the JWKS
+    {"email": None, "sub": None},            # no identity claim
+])
+def test_invalid_tokens_rejected(keypair, kw):
+    key, jwks = keypair
+    with pytest.raises(edge_auth.EdgeAuthError):
+        edge_auth.verify_cf_access_token(_token(key, **kw), team_domain=TEAM, aud=AUD,
jwks=jwks)
+
+
+def test_bad_signature_rejected(keypair):
+    """A token signed by a DIFFERENT key (but advertising kid=test-kid) must fail signature
check."""
+    _, jwks = keypair
+    other = rsa.generate_private_key(public_exponent=65537, key_size=2048)
+    with pytest.raises(edge_auth.EdgeAuthError):
+        edge_auth.verify_cf_access_token(_token(other), team_domain=TEAM, aud=AUD, jwks=jwks)
+
+
+# ---------------------------------------------------------------- resolve_tenant (default-
OFF)
+
+def test_resolve_tenant_off_is_noop_even_with_token(keypair):
+    """DEFAULT-OFF: edge_auth_enabled=False ? None, even when a (valid) token is present."""
+    key, _ = keypair
+    assert edge_auth.resolve_tenant(
+        {edge_auth.CF_ACCESS_HEADER: _token(key)}, _cfg(False)) is None
+
+
+def test_resolve_tenant_off_ignores_missing_header():
+    assert edge_auth.resolve_tenant({}, _cfg(False)) is None
+
+
+# ---------------------------------------------------------------- resolve_tenant (enabled)
+
+def test_resolve_tenant_on_valid(keypair):
+    key, jwks = keypair
+    user = edge_auth.resolve_tenant(
+        {edge_auth.CF_ACCESS_HEADER: _token(key)}, _cfg(True), jwks_fetcher=lambda td: jwks)
+    assert user == "u@example.com"
+
+
+def test_resolve_tenant_on_missing_header_raises(keypair):
+    _, jwks = keypair
+    with pytest.raises(edge_auth.EdgeAuthError):
+        edge_auth.resolve_tenant({}, _cfg(True), jwks_fetcher=lambda td: jwks)
+
+
+def test_resolve_tenant_enabled_but_unconfigured_raises(keypair):
+    key, _ = keypair
+    with pytest.raises(edge_auth.EdgeAuthError):
+        edge_auth.resolve_tenant({edge_auth.CF_ACCESS_HEADER: _token(key)}, _cfg(True, team="",
aud=""))
+
+
+def test_resolve_tenant_header_case_insensitive(keypair):
+    key, jwks = keypair
+    user = edge_auth.resolve_tenant(
+        {edge_auth.CF_ACCESS_HEADER.lower(): _token(key)}, _cfg(True), jwks_fetcher=lambda td:
jwks)
+    assert user == "u@example.com"
+
+
```

```
+# ------------------------------------------------------ /api/process wiring (owner_id tag)
+
+pytest.importorskip("httpx")  # fastapi TestClient transport
+
+
+def _client(tmp_path, monkeypatch, cfg):
+    from fastapi.testclient import TestClient
+
+    import backend.main as m
+    from backend.infrastructure.database import Database
+
+    db = Database(str(tmp_path / "t.db"))
+    monkeypatch.setattr(m, "db_instance", db)
+    monkeypatch.setattr(m, "global_config", cfg)
+
+    class _Noop:  # don't drain the job (we only assert the created row's owner_id)
+        def submit(self, fn, *a, **k):
+            return None
+
+    monkeypatch.setattr(m, "_get_executor", lambda: _Noop())
+    edge_auth._jwks_cache.clear()
+    return TestClient(m.app), db, m
+
+
+def test_process_default_off_tags_no_owner(tmp_path, monkeypatch):
+    """DEFAULT-OFF: /api/process needs no token and the job's owner_id is NULL (unchanged)."""
+    client, db, _m = _client(tmp_path, monkeypatch, AppConfig())  # edge_auth_enabled=False
+    v = tmp_path / "x.mp4"
+    v.write_bytes(b"x")
+    r = client.post("/api/process", json={"video_path": str(v)})
+    assert r.status_code == 202
+    assert db.get_job(r.json()["job_id"])["owner_id"] is None
+
+
+def test_process_edge_auth_missing_token_401(tmp_path, monkeypatch):
+    client, _db, _m = _client(tmp_path, monkeypatch, _cfg(True))
+    v = tmp_path / "x.mp4"
+    v.write_bytes(b"x")
+    r = client.post("/api/process", json={"video_path": str(v)})  # no Cf-Access header
+    assert r.status_code == 401
+
+
+def test_process_edge_auth_valid_token_sets_owner(tmp_path, monkeypatch, keypair):
+    key, jwks = keypair
+    client, db, m = _client(tmp_path, monkeypatch, _cfg(True))
+    monkeypatch.setattr(m.edge_auth, "_default_jwks_fetcher", lambda td: jwks)
+    v = tmp_path / "x.mp4"
+    v.write_bytes(b"x")
+    r = client.post("/api/process", json={"video_path": str(v)},
+                    headers={edge_auth.CF_ACCESS_HEADER: _token(key)})
+    assert r.status_code == 202
+    assert db.get_job(r.json()["job_id"])["owner_id"] == "u@example.com"
```

6. assistant (2026-06-21T13:40:13.870Z)

Now let me read the actual source files to verify the test claims against real behavior. Let me read the key files.

7. user (2026-06-21T13:40:14.626Z)

```
1       """Cloudflare Access edge-auth ? verify the Cf-Access JWT, derive the tenant ``user_id``
(M9, D9).
2
3       **DEFAULT-OFF:** when ``security.edge_auth_enabled`` is False the resolver is a strict
NO-OP (returns
```

```
4       ``None`` = the local single-user) ? nothing about the existing local path changes. When
enabled, the
5       ``Cf-Access-Jwt-Assertion`` header is **required** and **verified** (RS256) against the
team JWKS ?
6       signature + exp + issuer + audience ? so a direct hit to the Cloud Run URL **can't
spoof** the
7       identity header (the explicit D9 anti-spoof requirement). Returns the verified
``user_id`` (the
8       token's ``email``, else ``sub``).
9
10      ``jwt`` (``PyJWT[crypto]``) is **lazy-imported** so the base runtime works without it
(install the
11      ``auth`` extra for hosted/multi-tenant). The JWKS fetch is **injectable**, so the whole
verifier is
12      unit-tested **offline** with an in-test RSA keypair + a fake JWKS ? no network, no
Cloudflare.
13      """
14
15      from __future__ import annotations
16
17      import json
18      import time
19      import urllib.request
20      from typing import Any, Callable, Mapping, Optional
21
22      # Cloudflare Access sends the signed assertion in this header (it terminates at the CF
edge).
23      CF_ACCESS_HEADER = "Cf-Access-Jwt-Assertion"
24      _JWKS_PATH = "/cdn-cgi/access/certs"
25      _JWKS_TTL_SEC = 600.0  # re-fetch the team JWKS at most every 10 min (keys rotate
slowly)
26
27      JwksFetcher = Callable[[str], dict]
28
29
30      class EdgeAuthError(Exception):
31          """A required Cf-Access token was missing or failed verification ? the caller
returns 401/403."""
32
33
34      # module-level JWKS cache: team_domain -> (fetched_at, jwks_dict). Production only;
tests inject.
35      _jwks_cache: dict[str, tuple[float, dict]] = {}
36
37
38      def _issuer(team_domain: str) -> str:
39          return team_domain.rstrip("/")
40
41
42      def _default_jwks_fetcher(team_domain: str) -> dict:
43          """Fetch the team's public JWKS (``<team_domain>/cdn-cgi/access/certs``). Network ?
only the
44          production path reaches here; tests pass an injected fetcher."""
45          url = _issuer(team_domain) + _JWKS_PATH
46          with urllib.request.urlopen(url, timeout=5) as resp:  # noqa: S310 (https team
domain)
47              data: Any = json.load(resp)
48          if not isinstance(data, dict) or "keys" not in data:
49              raise EdgeAuthError(f"unexpected JWKS shape from {url}")
50          return data
51
52
53      def _get_jwks(team_domain: str, fetcher: Optional[JwksFetcher]) -> dict:
54          """Return the JWKS for ``team_domain``, cached with a TTL. An injected ``fetcher``
(tests)
```

```
55          bypasses the cache so each test is deterministic."""
56          if fetcher is not None:
57              return fetcher(team_domain)
58          now = time.time()
59          cached = _jwks_cache.get(team_domain)
60          if cached and (now - cached[0]) < _JWKS_TTL_SEC:
61              return cached[1]
62          jwks = _default_jwks_fetcher(team_domain)
63          _jwks_cache[team_domain] = (now, jwks)
64          return jwks
65
66
67      def verify_cf_access_token(token: str, *, team_domain: str, aud: str, jwks: dict) ->
str:
68          """Verify a Cf-Access JWT against ``jwks`` (RS256; checks sig + exp + issuer +
audience) and
69          return the user_id (``email`` else ``sub``). Raises ``EdgeAuthError`` on ANY
failure."""
70          try:
71              import jwt
72              from jwt import PyJWKSet
73          except Exception as exc:  # pragma: no cover - only when the auth extra isn't
installed
74              raise EdgeAuthError(
75                  "PyJWT[crypto] is required for edge auth ? install the 'auth' extra "
76                  "(pip install -e .[auth]) on a host with edge_auth_enabled."
77              ) from exc
78
79          try:
80              kid = jwt.get_unverified_header(token).get("kid")
81              if not kid:
82                  raise EdgeAuthError("token header has no kid")
83              signing_key = PyJWKSet.from_dict(jwks)[kid]
84              claims = jwt.decode(
85                  token, signing_key.key, algorithms=["RS256"],
86                  audience=aud, issuer=_issuer(team_domain),
87                  options={"require": ["exp", "iss", "aud"]},
88              )
89          except EdgeAuthError:
90              raise
91          except KeyError as exc:  # kid not in the JWKS
92              raise EdgeAuthError(f"no JWKS key for kid {exc}") from exc
93          except Exception as exc:  # jwt.* errors: bad sig / expired / wrong aud|iss /
malformed
94              raise EdgeAuthError(f"Cf-Access token verification failed: {exc}") from exc
95
96          user = claims.get("email") or claims.get("sub")
97          if not user:
98              raise EdgeAuthError("verified token carries no email/sub claim")
99          return str(user)
100
101
102     def resolve_tenant(headers: Mapping[str, str], config: Any, *,
103                        jwks_fetcher: Optional[JwksFetcher] = None) -> Optional[str]:
104         """Resolve the tenant ``user_id`` for a request, or ``None`` when edge auth is OFF.
105
106         DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-
user; unchanged).
107         Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403)
on any miss.
108         ``jwks_fetcher`` is injectable for offline tests.
109         """
110         sec = getattr(config, "security", None)
111         if not sec or not getattr(sec, "edge_auth_enabled", False):
112             return None  # DEFAULT-OFF ? no tenant identity, byte-identical to today
```

```
113
114         team_domain = getattr(sec, "cf_team_domain", "") or ""
115         aud = getattr(sec, "cf_access_aud", "") or ""
116         if not team_domain or not aud:
117             raise EdgeAuthError(
118                 "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not
configured")
119
120         # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but
support a plain
121         # dict too (tests) by trying both casings.
122         token = headers.get(CF_ACCESS_HEADER) or headers.get(CF_ACCESS_HEADER.lower())
123         if not token:
124             raise EdgeAuthError(f"missing {CF_ACCESS_HEADER} header (request did not pass
Cloudflare Access)")
125
126         jwks = _get_jwks(team_domain, jwks_fetcher)
127         return verify_cf_access_token(token, team_domain=team_domain, aud=aud, jwks=jwks)
128
```

8. assistant (2026-06-21T13:40:19.570Z)

Now let me look at the test wiring closely. The key question for test #3 (valid-token API test) is whether the monkeypatch is on the right target. The test monkeypatches `m.edge_auth._default_jwks_fetcher`, but `resolve_tenant` is called with `jwks_fetcher=None` in `main.py`, so `_get_jwks` calls `_default_jwks_fetcher`. Let me verify the call path and check whether PyJWKSet handles a `kid: None` advertised in JWKS, plus the reject reasons. Let me run the tests to see actual behavior.

9. user (2026-06-21T13:40:21.941Z)

```
PyJWT 2.13.0
crypto ok
httpx ok
```

10. user (2026-06-21T13:40:33.348Z)

```
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 17 items

tests/test_edge_auth.py::test_valid_token_returns_email PASSED           [  5%]
tests/test_edge_auth.py::test_falls_back_to_sub_when_no_email PASSED     [ 11%]
tests/test_edge_auth.py::test_invalid_tokens_rejected[kw0] PASSED        [ 17%]
tests/test_edge_auth.py::test_invalid_tokens_rejected[kw1] PASSED        [ 23%]
tests/test_edge_auth.py::test_invalid_tokens_rejected[kw2] PASSED        [ 29%]
tests/test_edge_auth.py::test_invalid_tokens_rejected[kw3] PASSED        [ 35%]
tests/test_edge_auth.py::test_invalid_tokens_rejected[kw4] PASSED        [ 41%]
tests/test_edge_auth.py::test_bad_signature_rejected PASSED              [ 47%]
tests/test_edge_auth.py::test_resolve_tenant_off_is_noop_even_with_token PASSED [ 52%]
tests/test_edge_auth.py::test_resolve_tenant_off_ignores_missing_header PASSED [ 58%]
tests/test_edge_auth.py::test_resolve_tenant_on_valid PASSED             [ 64%]
tests/test_edge_auth.py::test_resolve_tenant_on_missing_header_raises PASSED [ 70%]
tests/test_edge_auth.py::test_resolve_tenant_enabled_but_unconfigured_raises PASSED [ 76%]
tests/test_edge_auth.py::test_resolve_tenant_header_case_insensitive PASSED [ 82%]
tests/test_edge_auth.py::test_process_default_off_tags_no_owner PASSED   [ 88%]
tests/test_edge_auth.py::test_process_edge_auth_missing_token_401 PASSED [ 94%]
tests/test_edge_auth.py::test_process_edge_auth_valid_token_sets_owner PASSED [100%]
```

```
============================== 17 passed in 5.94s ==============================
```

11. assistant (2026-06-21T13:40:36.842Z)

All pass. Now the critical rigor question: do the reject-path tests fail for the *reason claimed*? The biggest risk is a test passing for an unrelated reason. Let me instrument each reject case to capture the actual `EdgeAuthError` message, so I can confirm e.g. the bad-sig test fails on signature (not kid), and the wrong-aud test fails on aud (not something earlier).

12. user (2026-06-21T13:40:45.426Z)

```
expired        -> Cf-Access token verification failed: Signature has expired
wrong-aud      -> Cf-Access token verification failed: Audience doesn't match
wrong-iss      -> Cf-Access token verification failed: Invalid issuer
unknown-kid    -> no JWKS key for kid 'keyset has no key for kid: unknown-kid'
no-claim       -> verified token carries no email/sub claim
bad-sig        -> Cf-Access token verification failed: Signature verification failed
```

13. assistant (2026-06-21T13:40:54.242Z)

Each reject case fails for the *exact reason claimed*:
- expired ? "Signature has expired"
- wrong-aud ? "Audience doesn't match"
- wrong-iss ? "Invalid issuer"
- unknown-kid ? "keyset has no key for kid"
- no-claim ? "verified token carries no email/sub claim"
- bad-sig ? "Signature verification failed" (genuinely exercises signature failure, NOT kid ? kid resolves fine to test-kid, then RS256 sig fails against the real public key)

This is rigorous. The bad-sig test (Q2) truly exercises signature failure. Now let me verify the order of checks ? is there any case where an earlier check could mask the intended one? The concern: does `jwt.decode` check signature before aud/iss/exp? If so, a wrong-aud token signed by the real key still passes sig, so aud is genuinely tested. Let me also check the ordering for a more subtle scenario and confirm the no-claim test isn't passing because of a sig/decode failure instead.

The no-claim case shows the token decoded successfully (got past sig/exp/aud/iss) and failed at the email/sub check ? exactly right.

Now let me verify the API tests. Test #3 concern: is the 401 in the missing-token test from auth or from validation? In `main.py`, `resolve_tenant` runs *before* `validate_input_path`. Let me confirm a valid file is written so a 400 path isn't accidentally hit, and confirm the monkeypatch target for the valid-token test.

14. user (2026-06-21T13:41:01.851Z)

```
resolve_tenant before validate_input_path: True
```

15. assistant (2026-06-21T13:41:06.473Z)

Auth runs first. But there's a subtle non-vacuity concern for the missing-token 401 test: it writes a valid `x.mp4` file. Even if auth were somehow bypassed, the request would proceed ? but since auth is first and the config has edge_auth ON with no header, the 401 is genuinely from auth. Let me confirm the 401 is from auth and not coincidental by checking the response detail, and also probe whether the missing-token test would still be 401 if the file *didn't* exist (to ensure non-vacuity isn't masked). Let me also verify the valid-token monkeypatch path actually avoids network.

16. user (2026-06-21T13:41:22.252Z)

```
19:11:21 [INFO] httpx: HTTP Request: POST http://testserver/api/process "HTTP/1.1 401
Unauthorized"
19:11:21 [INFO] httpx: HTTP Request: POST http://testserver/api/process "HTTP/1.1 202 Accepted"
missing-token (no file even): 401 edge auth: missing Cf-Access-Jwt-Assertion header (request did
```

```
not pass Cloudflare Access)
valid-token: 202 owner: u@example.com
```

17. assistant (2026-06-21T13:41:30.566Z)

```
The 401 detail confirms it's `edge auth:` (non-vacuous ? auth-derived, not validation). The
valid-token test correctly monkeypatches `_default_jwks_fetcher`, and the call path through
`resolve_tenant(jwks_fetcher=None)` ? `_get_jwks` ? `_default_jwks_fetcher` works without
network.

Now, Q3/Q4 ? the **core anti-spoof test (a different-but-valid signature from a key NOT in the
JWKS)**. The `test_bad_signature_rejected` test covers exactly this at the *verifier* level: a
token signed by `other` key (valid signature, but the public key is not in JWKS) is rejected.
Confirmed above.

However, let me note a gap for Q4: this anti-spoof is **not** exercised at the `/api/process`
(D9 endpoint) level ? only the verifier unit. Let me also check whether there's a test for the
resolver-enabled path with a *bad* token (wrong sig) going through `resolve_tenant`. The reject
paths only go through `verify_cf_access_token` directly. Let me confirm the resolver wraps the
verifier so the reject-path coverage transfers (it does via `_get_jwks` ?
`verify_cf_access_token`).

Now let me check the hygiene items: ruff, mypy, and the `pytest.importorskip` placement /
module-scoped fixture reuse.
```

18. user (2026-06-21T13:41:32.171Z)

```
All checks passed!
```

19. user (2026-06-21T13:41:36.833Z)

```
Success: no issues found in 1 source file
=== mypy config ===
[mypy]
```

Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean modules and pins an explicit quarantine list below ? not a typing crusade. Ratchet: when you touch a quarantined module, try removing its entry.

```
ignore_missing_imports = True
```

backend/ is a namespace package (no root __init__.py) ? required so mypy maps files to backend.* module names instead of duplicating top-level names.

```
explicit_package_bases = True
```

--- Quarantine (known-loose modules, each with a reason) ---

Optional module globals (db_instance/global_config set only in main()); properly fixed by the deferred app-factory/lifespan refactor (BACKLOG).

```
[mypy-backend.main]
ignore_errors = True
```

Job-worker drain/wake loop extracted from backend.main (same Optional module-glob debt: resolves db_instance/global_config THROUGH backend.main at call-time). Share backend.main's quarantine until the deferred app-factory/lifespan refactor lands.

```
[mypy-backend.api.job_worker]
ignore_errors = True
```

Worker dispatcher: composes the SAME validator/segmenter/report building blocks a

backend.main, which are typed `config: dict` but receive the AppConfig migration shim at runtime (the .get() compat layer). Shares backend.main's config-shim quarantine until the typed-config migration removes the dict?AppConfig duality everywhere.

```
[mypy-backend.worker.__main__]
ignore_errors = True
```

Legacy demo segmenter (fabricated metadata; coverage-omitted too).

```
[mypy-backend.pipeline.segmenters.motion]
ignore_errors = True
```

yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code to detectors/person_detector.py (which IS type-clean, no quarantine). What remains here is pre-existing orchestration debt unrelated to that move (implicit-Optional process_video defaults, the Success|Failure ABC-override mismatch, a Future[...] annotation) ? ratchet those out in a focused typing pass, not here. (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)

```
[mypy-backend.pipeline.segmenters.yolo_hybrid]
```

20. assistant (2026-06-21T13:41:42.564Z)

auth.py is mypy-clean and ruff-clean. Now let me check the test file's `pytest.importorskip` placements and the module-scoped fixture. There are two importorskip calls: one at module top (`jwt`) and one mid-file (`httpx`). The mid-file one at line ~131 is unusual. Let me verify it actually guards correctly and check if there's a concern about the module-scoped `keypair` fixture being reused/mutated across tests.

21. user (2026-06-21T13:41:43.398Z)

```
120
121     def test_resolve_tenant_header_case_insensitive(keypair):
122         key, jwks = keypair
123         user = edge_auth.resolve_tenant(
124             {edge_auth.CF_ACCESS_HEADER.lower(): _token(key)}, _cfg(True),
jwks_fetcher=lambda td: jwks)
125         assert user == "u@example.com"
126
127
128     # ------------------------------------------------------- /api/process wiring (owner_id
tag)
129
130     pytest.importorskip("httpx")  # fastapi TestClient transport
131
132
133     def _client(tmp_path, monkeypatch, cfg):
134         from fastapi.testclient import TestClient
135
136         import backend.main as m
137         from backend.infrastructure.database import Database
138
139         db = Database(str(tmp_path / "t.db"))
140         monkeypatch.setattr(m, "db_instance", db)
141         monkeypatch.setattr(m, "global_config", cfg)
142
143         class _Noop:  # don't drain the job (we only assert the created row's owner_id)
144             def submit(self, fn, *a, **k):
145                 return None
146
147         monkeypatch.setattr(m, "_get_executor", lambda: _Noop())
```

```
148         edge_auth._jwks_cache.clear()
149         return TestClient(m.app), db, m
150
151
152     def test_process_default_off_tags_no_owner(tmp_path, monkeypatch):
153         """DEFAULT-OFF: /api/process needs no token and the job's owner_id is NULL
(unchanged)."""
154         client, db, _m = _client(tmp_path, monkeypatch, AppConfig())  #
edge_auth_enabled=False
155         v = tmp_path / "x.mp4"
156         v.write_bytes(b"x")
157         r = client.post("/api/process", json={"video_path": str(v)})
158         assert r.status_code == 202
159         assert db.get_job(r.json()["job_id"])["owner_id"] is None
160
161
162     def test_process_edge_auth_missing_token_401(tmp_path, monkeypatch):
163         client, _db, _m = _client(tmp_path, monkeypatch, _cfg(True))
164         v = tmp_path / "x.mp4"
165         v.write_bytes(b"x")
166         r = client.post("/api/process", json={"video_path": str(v)})  # no Cf-Access header
167         assert r.status_code == 401
168
169
170     def test_process_edge_auth_valid_token_sets_owner(tmp_path, monkeypatch, keypair):
171         key, jwks = keypair
172         client, db, m = _client(tmp_path, monkeypatch, _cfg(True))
173         monkeypatch.setattr(m.edge_auth, "_default_jwks_fetcher", lambda td: jwks)
174         v = tmp_path / "x.mp4"
175         v.write_bytes(b"x")
176         r = client.post("/api/process", json={"video_path": str(v)},
177                         headers={edge_auth.CF_ACCESS_HEADER: _token(key)})
178         assert r.status_code == 202
179         assert db.get_job(r.json()["job_id"])["owner_id"] == "u@example.com"
```

22. assistant (2026-06-21T13:41:49.016Z)

Now let me check one subtle non-vacuity concern about the **valid-token API test** (test #3). It monkeypatches `m.edge_auth._default_jwks_fetcher`. But note: `_get_jwks` caches into `_jwks_cache` when `fetcher is None`. The `_client` helper clears `_jwks_cache`, but the monkeypatch in the valid-token test happens *after* `_client` returns. Since these are separate tests, cache pollution across tests is the concern. Let me verify the cache isn't polluting ? and more importantly, verify the **default-off API test is genuinely non-vacuous** (could it pass even if owner_id wiring were broken?). Let me check `get_job` returns the `owner_id` column and that the schema has it.

23. user (2026-06-21T13:41:50.455Z)

```
300:             ("owner_id", "TEXT"),               # who pays (the non-owner user / tenant)
313:         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
314:                        "WHERE owner_id IS NOT NULL")
542:                    owner_id: Optional[str] = None) -> bool:
544:         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
548:             "owner_id) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?)",
551:              max_frames, json.dumps(policy) if policy is not None else None, owner_id),
626:                        owner_id: Optional[str] = None, margin: float = 0.0) ->
Optional[Dict[str, Any]]:
650:             "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?,
gpu_active_seconds=?, "
653:             (owner_id, compute_backend, gpu_type, usage.get(billing.GPU_SECONDS),
667:                 "SELECT id, video_id, owner_id, compute_backend, gpu_type,
gpu_active_seconds, "
```

24. assistant (2026-06-21T13:41:56.636Z)

The `owner_id` column already existed in the schema (line 300, with comment "who pays" ? added in an earlier milestone for `claim_due_job`). So `create_job` previously inserted rows *without* setting `owner_id`, leaving it NULL by default. This is important for the non-vacuity of `test_process_default_off_tags_no_owner`: a NULL `owner_id` is the SQLite default for an unset column anyway. Let me verify this ? if a row created the OLD way (without the owner_id column in the INSERT) also yields NULL, then the default-off test asserting `owner_id is None` would pass *even if the new `owner_id=owner_id` wiring were entirely absent*. That makes the default-off test partially vacuous as a regression guard for the wiring.

Let me confirm: the default-off test passes `owner_id=None` anyway (resolver returns None), so it can never distinguish "wiring present passing None" from "wiring absent". The real wiring guard is the valid-token test. Let me verify the default-off test cannot catch a broken wiring.

25. user (2026-06-21T13:41:58.192Z)

```
    # worker logs loudly on a failed transition so a job can't silently wedge.

    def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
                   player_pool: Optional[List[str]] = None,
                   max_frames: Optional[int] = None,
                   policy: Optional[Dict[str, Any]] = None) -> bool:
        """Insert a new job in state 'queued'. ``policy`` is an optional JSON ExecutionPolicy
        (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
        return self._execute_write(
            "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames, status,
policy) "
            "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
            (job_id, video_path, segmenter,
             json.dumps(player_pool) if player_pool is not None else None,
             max_frames, json.dumps(policy) if policy is not None else None),
            f"Failed to create job {job_id}",
        )

    def mark_job_running(self, job_id: str) -> bool:
        return self._execute_write(
            "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
            "progress='starting' WHERE id = ?", (job_id,),
```

26. assistant (2026-06-21T13:42:08.219Z)

Confirmed. Before this change, `create_job` did not set `owner_id`, and the column defaulted to NULL. So `test_process_default_off_tags_no_owner` asserting `owner_id is None` would pass identically whether the new wiring exists or not ? it's a weak regression guard for the wiring (though it does correctly verify the DEFAULT-OFF contract that no token is required and a 202 is returned). The genuine wiring guard is the valid-token test, which is non-vacuous (asserts the specific email). This is a minor weakness, not a blocker.

Now let me check the JWKS cache-pollution hygiene concern. The valid-token API test uses the real (uninjected) path: `resolve_tenant(..., jwks_fetcher=None)` ? `_get_jwks(None)` ? caches into `_jwks_cache["https://team.cloudflareaccess.com"]`. The `_client` helper clears the cache at setup. But the order matters: if `test_process_edge_auth_valid_token_sets_owner` populates the cache, a *later* test (none here, but future) hitting the same team domain with `jwks_fetcher=None` could get a stale entry. Within this file it's fine because `_client` clears it. Let me check the verifier-level reject tests don't leak into the cache (they call `verify_cf_access_token` directly, bypassing cache ? fine).

Let me verify the module-scoped `keypair` fixture isn't mutated by any test (it returns a dict that tests could theoretically mutate). The `_token` helper and `jwk.update` happen inside the fixture once; tests only read. Confirmed by passing runs. Let me run the full relevant test subset plus check the `Request` import is actually used and no broken behavior in CORS import-time load.

27. user (2026-06-21T13:42:11.462Z)

```
jwks mutations: [] [] []
```

28. assistant (2026-06-21T13:42:16.464Z)

No mutations of the shared `jwks` ? module-scoped fixture reuse is safe. Now let me check one more rigor angle: is there an `alg`-confusion / `none`-alg defense test? The verifier hardcodes `algorithms=["RS256"]`, so an `alg:none` token would be rejected, but there's no test for it. Also no test that a token with `alg:HS256` (using the public key as HMAC secret ? classic JWT confusion attack) is rejected. This is a notable missing anti-spoof test given D9's emphasis. Let me verify the code is actually safe against it (it is, since algorithms is pinned) and confirm there's no test.

29. user (2026-06-21T13:42:18.050Z)

```
15:from jwt.algorithms import RSAAlgorithm  # noqa: E402
42:    return jwt.encode(claims, key, algorithm="RS256", headers={"kid": kid})
--- code pins: ---
85:            token, signing_key.key, algorithms=["RS256"],
```

30. assistant (2026-06-21T13:42:23.839Z)

No alg-confusion test exists. The code is safe (pins `["RS256"]`), but for a security-critical D9 anti-spoof boundary, the absence of an `alg:none` / `HS256`-confusion rejection test is a genuine coverage gap worth noting (these are the canonical JWT bypass attacks).

Now let me verify the `pytest.importorskip("httpx")` mid-module placement (line 130) works correctly ? it's after the verifier/resolver tests, so if httpx is absent, only the API tests are skipped while verifier tests still run. That's actually good design (graceful degradation). But the placement is unusual ? let me confirm it doesn't skip the *whole* module. From the earlier run, all 17 ran, so httpx is present. The placement is fine and intentional.

Let me run the broader test suite quickly to confirm nothing regressed from the `create_job` signature change and the import-time CORS load.

31. user (2026-06-21T13:42:51.416Z)

```
.................................................................. [ 53%]
.........................................................          [100%]
134 passed, 1072 deselected in 24.45s
```

32. assistant (2026-06-21T13:42:57.643Z)

No regressions. Let me do a final check on the `_token` helper's `sub` handling for the no-claim parametrize case to make sure that test path is genuinely the email/sub-empty branch and not an earlier failure (already confirmed via the message capture: "verified token carries no email/sub claim").

Let me also verify the `unknown-kid` reject is distinct from the bad-sig reject ? they must be testing different code branches. The unknown-kid raises via `KeyError`/`no JWKS key for kid`, while bad-sig raises via `Signature verification failed`. Confirmed distinct from the message capture.

One last hygiene check: confirm the test file has no unused imports flagged and `from backend.config import AppConfig` resolves (used in `_cfg` and the default-off test).

33. user (2026-06-21T13:42:59.123Z)

```
E501 Line too long (98 > 88)
 --> tests\test_edge_auth.py:3:89
  |
1 | """Cloudflare Access edge-auth (COMPUTE_DECOUPLED_SERVING M9, D9).
2 |
```

```
3 | Fully OFFLINE: an in-test RSA keypair signs Cf-Access JWTs + builds a fake JWKS, so the
verifier +
  |
^^^^^^^^^^
4 | the tenant resolver are exercised with no network and no Cloudflare. Cardinal invariant:
with
5 | ``edge_auth_enabled=False`` the resolver is a strict NO-OP (returns None = today's local
single-user)
  |
exit=0
```

34. assistant (2026-06-21T13:43:02.586Z)

The `--select E` flag surfaces E501 (line-length) which the repo's default ruff config evidently ignores (the repo-config run earlier passed clean). Let me confirm the repo's ruff line-length setting so I don't report a false positive ? the project config governs CI, not my ad-hoc `--select`.

35. user (2026-06-21T13:43:04.763Z)

```
ruff.toml:16:[lint.per-file-ignores]
=== default repo ruff run ===
All checks passed!
```

36. user (2026-06-21T13:43:07.226Z)

```
1       # Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would
2       # bury blame history for zero behavior value).
3       #
4       # Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle
5       # warnings (W) + bugbear (B). Deferred with intent: I (import-sort) and UP
6       # (pyupgrade) ? both are large mechanical diffs; adopt via a dedicated PR.
7       target-version = "py38"
8
9       # scratch/ = throwaway debugging scripts that may hit real APIs; it is also
10      # excluded from pytest collection (see pytest.ini).
11      extend-exclude = ["scratch"]
12
13      [lint]
14      extend-select = ["W", "B"]
15
16      [lint.per-file-ignores]
17      # Entrypoint scripts deliberately run setup (load_dotenv, logging config,
18      # sys.path) before importing backend modules ? import order is load-bearing.
19      "backend/main.py" = ["E402"]
20      "scripts/run_phase3_eval.py" = ["E402"]
21      "scripts/run_process_and_stitch.py" = ["E402"]
22
```

37. assistant (2026-06-21T13:43:15.108Z)

The repo ruff config has no line-length setting, so E501 is not in the default selection ? the repo deliberately does not enforce line length. So the E501 I surfaced with `--select E` is NOT a real CI finding. The default repo run (`ruff check`, matching CI) passes clean. No ruff finding to report.

I have thoroughly verified all five lens questions. Let me also confirm the `target-version = "py38"` vs the `dict[str, tuple[...]]` builtin-generic annotations in auth.py (line 35) ? under py3.8 `dict[...]` as a runtime annotation would fail, but `from __future__ import annotations` (line 15) makes all annotations strings, so it's fine at runtime. The module-level `_jwks_cache: dict[...]` at line 35 is a module-level *variable annotation* ? with `from __future__ import annotations` that's also a string, not evaluated. Good. Ruff py38 target passed clean, confirming.

Let me compile my findings.

38. user (2026-06-21T13:43:33.968Z)

Structured output provided successfully

39. assistant (2026-06-21T13:43:46.711Z)

I completed the TEST RIGOR / HYGIENE review of the M9 edge-auth change (HEAD 03077c3) and returned findings via StructuredOutput.

Summary of what I verified (all grounded by running the code, not just reading it):

- **Reject-path reasons (Q1):** I instrumented every reject case and confirmed each fails for the exact claimed reason (expired?"Signature has expired", wrong-aud?"Audience doesn't match", wrong-iss?"Invalid issuer", unknown-kid?"keyset has no key for kid", no-claim?"verified token carries no email/sub claim", bad-sig?"Signature verification failed"). None pass for an unrelated reason.
- **bad-sig test (Q2):** Genuinely exercises signature failure ? kid resolves to test-kid fine, then RS256 sig fails against the real public key. It IS the core anti-spoof case (Q4) at the verifier level (a valid-but-foreign-key signature is rejected).
- **API tests (Q3):** Non-vacuous. The missing-token 401 is auth-derived (detail "edge auth: missing Cf-Access..."), auth runs before path validation, and the valid-token test correctly monkeypatches `_default_jwks_fetcher` ? I sabotaged the network fetcher to raise and confirmed no network is hit.
- **Hygiene:** `ruff check` (repo config, matching CI) and `mypy` both clean on `backend/api/auth.py`; the repo intentionally doesn't enforce line-length so the E501 my ad-hoc `--select E` surfaced is not a real CI finding. `from __future__ import annotations` makes the `dict[...]` py38 annotations safe. The mid-module `pytest.importorskip("httpx")` placement is intentional and correct (verifier tests still run if httpx is absent). The module-scoped `keypair` fixture is never mutated (read-only across tests). Full relevant suite green (134 passed, no regression from the `create_job` signature change or import-time CORS load).

Two real findings (both nit-level, no blocker):

1. `tests/test_edge_auth.py:152-159` ? the default-OFF owner-NULL test is a weak regression guard for the new `owner_id` wiring: `jobs.owner_id` already existed and defaulted to NULL, and the pre-M9 `create_job` didn't name it, so the assertion passes identically with or without the new `owner_id=owner_id` plumbing. The genuine wiring guard is the valid-token test. It still correctly verifies the DEFAULT-OFF contract.
2. `tests/test_edge_auth.py:60-71` ? no `alg='none'` / HS256-confusion rejection test. The code is safe (`algorithms=["RS256"]` pinned at `auth.py:85`), but for this D9 anti-spoof boundary the two canonical JWT bypass attacks are the highest-value adversarial cases and are untested.